

LEVEL 2 — NumPy Array Properties

Example Array

```
import numpy as np

arr = np.array([[10, 20, 30],
                [40, 50, 60]])
```

The array looks like this:

```
[[10 20 30]
 [40 50 60]]
```

1. ndim — Number of Dimensions

```
print(arr.ndim)
```

Output: 2

Meaning: The array has 2 dimensions. It has rows and columns.

2. shape — Size of Each Dimension

```
print(arr.shape)
```

Output: (2, 3)

Meaning: The array has 2 rows and 3 columns.

3. size — Total Number of Elements

```
print(arr.size)
```

Output: 6

Meaning: There are 6 values in the array.

4. dtype — Data Type of Elements

```
print(arr.dtype)
```

Typical output: int64 (may be int32 on some systems)

Meaning: It tells you the data type used to store the values.

5. itemsize — Bytes Used by One Element

```
print(arr.itemsize)
```

Typical output: 8 for int64

Meaning: Each element uses 8 bytes when the dtype is int64.

6. nbytes — Total Bytes Used by Array Data

```
print(arr.nbytes)
```

Typical output: 48 for 6 int64 elements

Meaning: Total memory used by the array's elements. Formula: $\text{size} \times \text{itemsize}$.

7. T — Transpose of the Array

```
print(arr.T)
```

Output:

```
[[10 40]
 [20 50]
 [30 60]]
```

Meaning: Rows become columns and columns become rows.

8. flags — Information About the Array

```
print(arr.flags)
```

Meaning: flags displays information about how the array is stored and accessed, such as whether it is C-contiguous, writable, and aligned.

Complete Code

```
import numpy as np

arr = np.array([[10, 20, 30],
                [40, 50, 60]])

print(arr.ndim)
print(arr.shape)
print(arr.size)
print(arr.dtype)
print(arr.itemsize)
print(arr.nbytes)
print(arr.T)
print(arr.flags)
```

Quick Revision

ndim	→ Number of dimensions
shape	→ Rows and columns / size of each dimension
size	→ Total number of elements
dtype	→ Data type of elements
itemsize	→ Bytes used by one element
nbytes	→ Total bytes used by array data
T	→ Transpose
flags	→ Array storage/access information